38.4 故障排查

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~280k tokens，输出 ~5k tokens（本节）

即便是精心设计的系统也会遇到问题。OpenClaw 的 `doctor` 命令提供了一站式的自动化诊断和修复工具，覆盖了从配置错误到守护进程故障的广泛场景。

38.4.1 Doctor 命令架构

`openclaw doctor` （ `src/commands/doctor.ts` ）是一个模块化的诊断框架，由 20+ 个专门的检查模块组成：

```
openclaw doctor
    │
    ├── doctor-update.ts         — 版本更新检查
    ├── doctor-ui.ts             — UI 协议版本兼容性
    ├── doctor-install.ts        — 源码安装问题检测
    ├── doctor-platform-notes.ts — 平台特定问题（macOS launchd 覆盖等）
    ├── doctor-config-flow.ts    — 配置文件迁移与修复
    ├── doctor-auth.ts           — 认证配置（OAuth、API 密钥）
    ├── doctor-legacy-config.ts  — 旧配置格式检测
    ├── doctor-state-integrity.ts — 状态目录完整性
    ├── doctor-state-migrations.ts — 状态数据迁移
    ├── doctor-sandbox.ts        — 沙箱镜像检查
    ├── doctor-security.ts       — 安全审计
    ├── doctor-gateway-services.ts — Gateway 服务配置
    ├── doctor-gateway-daemon-flow.ts — 守护进程管理
    ├── doctor-gateway-health.ts — Gateway 健康探测
    ├── doctor-workspace.ts      — 工作空间检查
    ├── doctor-workspace-status.ts — 工作空间状态
    ├── doctor-completion.ts     — Shell 自动补全
    └── doctor-prompter.ts       — 交互式提示器
```

执行流程

Doctor 的检查顺序经过精心设计——从最基础的到最高级的，每一步都可能修复后续步骤的前置条件：

1. **版本检查**：是否有可用更新，可选自动更新
2. **UI 兼容性**：控制台 UI 版本是否与后端匹配
3. **安装检查**：源码安装是否完整
4. **平台检测**：macOS Launch Agent 覆盖、环境变量等
5. **配置迁移**：旧版配置格式升级
6. **认证修复**：Gateway token/password 配置
7. **OAuth 修复**：Anthropic OAuth profile 迁移
8. **状态迁移**：旧版状态目录升级
9. **状态完整性**：检查损坏的会话文件
10. **沙箱检查**：Docker 镜像是否存在和最新
11. **服务配置**：launchd/systemd 服务文件
12. **安全审计**：敏感配置检查
13. **模型配置**：hooks 模型引用有效性
14. **Gateway 健康**：主动探测运行中的 Gateway
15. **守护进程**：启动/重启 Gateway 服务
16. **工作空间**：memory system 建议
17. **Shell 补全**：配置 bash/zsh 自动补全

两种运行模式

```
openclaw doctor          # 交互模式：逐步提示是否修复
openclaw doctor --fix    # 自动修复模式：跳过确认直接修复
```

交互模式使用 `@clack/prompts` 提供漂亮的终端 UI，每项检查发现问题后询问用户是否修复。 `--fix` 模式（ `nonInteractive: true` ）则自动同意所有修复建议，适用于 CI/CD 环境。

38.4.2 关键检查项详解

Gateway 认证修复

Doctor 检查 Gateway 认证配置是否安全：

```
const auth = resolveGatewayAuth({
  authConfig: cfg.gateway?.auth,
  tailscaleMode: cfg.gateway?.tailscale?.mode ?? "off",
});
const needsToken = auth.mode !== "password" &&
  (auth.mode !== "token" || !auth.token);

if (needsToken) {
  // 提示生成随机 token
  const nextToken = randomToken();  // 生成安全的随机令牌
  cfg.gateway.auth = { mode: "token", token: nextToken };
}
```

即使 Gateway 绑定在 loopback，也推荐启用 token 认证——这防止同一台机器上的其他进程未经授权访问 Gateway。

安全审计

`doctor-security.ts` 检查多种安全风险：

- Gateway 暴露到公网但没有认证
- 控制台 UI 通过反向代理暴露但 `trustedProxies` 未配置
- Tailscale Funnel 没有密码认证
- 敏感环境变量泄露

沙箱镜像检查

`doctor-sandbox.ts` 验证 Docker 沙箱镜像的可用性：

- 检查 `openclaw/sandbox:latest` 是否存在
- 如果不存在，提示拉取
- 检查镜像是否过旧，建议更新
- 检查沙箱作用域配置的一致性

守护进程管理

`doctor-gateway-daemon-flow.ts` 处理 Gateway 服务的启动和重启：

- 检测当前平台的服务管理器（macOS: launchd, Linux: systemd）
- 验证服务文件是否存在且配置正确
- 如果 Gateway 没有运行，提示启动

- 如果配置有变更，提示重启

38.4.3 日志系统

OpenClaw 使用分层日志系统：

日志级别

级别	用途	启用方式
error	错误和异常	始终启用
warn	警告信息	始终启用
info	一般信息	始终启用
debug	调试信息	`OPENCLAW_VERBOSE=1`
verbose	详细追踪	`OPENCLAW_VERBOSE=1`

子日志器

Gateway 为不同子系统创建独立的子日志器（child logger），便于按模块过滤：

```
// src/gateway/server.impl.ts
const logTailscale = log.child("tailscale");
const logBrowser = log.child("browser");
const logDiscovery = log.child("discovery");
const logHealth = log.child("health");
```

常见问题诊断

症状	可能原因	诊断命令
Gateway 无法启动	端口被占用	`lsof -i :18789`
WebSocket 连接失败	认证错误	`openclaw doctor`
聊天无响应	API 密钥无效	`openclaw health --verbose`
频道消息不收发	频道未配置/未链接	`openclaw health`

症状	可能原因	诊断命令
macOS 启动后无 Gateway	launchd 配置缺失	`launchctl list | grep openclaw`
Linux 重启后无 Gateway	linger 未启用	`loginctl show-user $USER | grep Linger`
沙箱命令超时	Docker 未运行/镜像缺失	`docker ps` + `openclaw doctor`
UI 加载空白	UI 资源版本不匹配	`openclaw doctor` （检查 UI 协议版本）

调试环境变量

变量	作用
`OPENCLAW_VERBOSE=1`	启用详细日志
`OPENCLAW_DEBUG_HEALTH=1`	健康检查调试输出
`OPENCLAW_DEBUG=1`	通用调试模式
`DEBUG=openclaw:*`	Node.js debug 模块过滤

38.4.4 状态恢复

当 OpenClaw 的状态数据（`~/.openclaw/`）损坏时，Doctor 提供了恢复路径：

1. **状态完整性检查**（`doctor-state-integrity.ts`）：扫描会话文件，检测损坏的 JSON
2. **状态迁移**（`doctor-state-migrations.ts`）：从旧版目录（`.clawdbot`、`.moltbot`）迁移数据
3. **工作空间备份提示**（`doctor-workspace-status.ts`）：建议用户定期备份工作空间
4. **配置回退**：如果 `openclaw.json` 损坏，Doctor 会尝试从 `.bak` 备份恢复

本节小结

1. **`openclaw doctor`** 是一个模块化诊断框架，包含 20+ 个检查模块，按依赖顺序从基础到高级执行，支持交互式和自动修复两种模式。

2. **关键检查**覆盖认证安全、沙箱镜像、守护进程、服务配置、状态完整性等多个维度，每项检查都能自动提供修复建议。
3. **日志系统**采用分层设计，支持按模块过滤的子日志器，环境变量控制调试级别。
4. **状态恢复**提供了从损坏数据恢复的完整路径：完整性扫描 → 旧版迁移 → 备份恢复。
5. 当遇到无法自动修复的问题时，Doctor 会输出具体的手动修复命令（如 `openclaw config set ...`），降低用户排错门槛。